

**Dissertation Title: Semantic Reconstruction of Software
Intent: A CloudHosted Agentic AI Solution for Study of
Requirement Traceability in Production Grade Codebases**

**Course No. : CCZG628T
Course Title: Dissertation**

**Dissertation Done by:
Student Name: Aarya Nanndaann Singh M N
BITS ID: 2024MT03013**

**Dissertation carried out at:
Aqueralabs India Pvt Ltd, Bangalore 560066**

**Submitted in partial fulfilment of M-Tech in Cloud
Computing degree programme**

**Research Area: Artificial Intelligence and Cloud
Computing**

**Under the Supervision of
Hanumanthu Indrakanti
Aqueralabs India Pvt Ltd, Bangalore 560066**



**BIRLA INSTITUTE OF TECHNOLOGY &
SCIENCE, PILANI
VIDYA VIHAR, PILANI, RAJASTHAN - 333031.**

March - 2026

Content

	Page No
Abstract	03
Introduction	04
Problem Statement	05
Dissertation Objectives	06
System Architecture	07
Progress of Prototype Implementation	11
Plan of Work	14
Observation	15
Challenges	16
Conclusion	18

Abstract

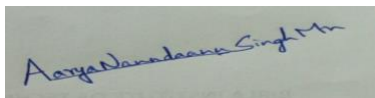
Modern enterprise software systems suffer from a critical challenge of *semantic intent loss*, where the underlying rationale behind implementation decisions becomes fragmented across multiple development artifacts such as issue tracking systems, commit histories, pull requests, and internal documentation. While source code effectively captures *what* a system does, it fails to preserve *why* specific changes were introduced, leading to increased cognitive load during onboarding, debugging, and maintenance activities. This problem is particularly pronounced in large-scale production codebases and open-source ecosystems, where contextual knowledge is distributed and often undocumented.

To address this challenge, this dissertation proposes a **cloud-hosted agentic AI framework** for the semantic reconstruction of software intent. The system is designed to act as an intelligent intermediary between developers and enterprise DevOps ecosystems by integrating with platforms such as JIRA and Github. It leverages multi-agent reasoning and large language models to establish requirement-to-code traceability, infer implementation intent, and generate contextual explanations of how and why specific code changes resolve requirements or defects.

At the midterm stage, significant progress has been made toward realizing the proposed system. The initial architecture has been designed, and the **artifact integration layer** has been implemented to extract and process data from issue tracking systems and version control repositories. Preliminary pipelines for correlating tickets, commits, and code changes have been developed.

The current prototype demonstrates the capability to ingest development artifacts which can provide **basic contextual associations between requirements and code changes**, serving as an initial step toward intent reconstruction. However, advanced reasoning capabilities, conversational explanation generation, and full IDE integration are still under development. The ongoing work aims to enhance the system with multi-agent orchestration, deeper semantic inference, and validation mechanisms for reproducing and testing issue resolutions.

This midterm progress establishes the feasibility of an intent-aware AI system for software engineering and lays the groundwork for subsequent phases involving reasoning enhancement, evaluation, and real-world deployment.



Signature of the Student

Name: Aarya Nanndaann Singh M N

Date: 25 March 2026

Place: Bangalore



Signature of the Supervisor

Name: Hanumanthu Indrakanti

Date: 25 March 2026

Place: Bangalore

Introduction

Modern software systems have evolved into highly complex, distributed, and continuously evolving ecosystems, driven by rapid development cycles and collaborative DevOps practices. In such environments, the software development lifecycle is no longer confined to source code alone, but is instead distributed across multiple interconnected artifacts, including issue tracking systems, version control repositories, pull request discussions, and continuous integration pipelines. While these artifacts collectively capture the evolution of a system, they introduce a critical challenge known as the **semantic gap between implementation and intent**.

This semantic gap refers to the disconnect between “*what the code does* and *why the code was written in a particular manner*”. Source code provides a precise description of system behaviour, but it often lacks explicit representation of the underlying rationale, such as business requirements, design decisions, bug fixes, or architectural constraints. Over time, this leads to **intent erosion**, where the original motivations behind code changes become obscured or lost, significantly impacting maintainability, debugging efficiency, and knowledge transfer within development teams.

In enterprise environments, this problem is further amplified due to the reliance on distributed DevOps ecosystems. Developers frequently depend on platforms such as **JIRA for requirement and issue tracking, Bitbucket, Github for version control and code collaboration**, and various CI/CD tools for deployment and validation workflows. Although these systems contain valuable contextual information, the knowledge is inherently fragmented and lacks a unified semantic representation. As a result, developers are required to manually traverse multiple sources to understand the relationship between requirements and code, leading to increased cognitive load and reduced productivity.

This challenge is particularly evident in scenarios such as onboarding new engineers, performing defect triaging, understanding legacy systems, and contributing to open-source projects. In such cases, the inability to quickly reconstruct the *intent* behind implementation decisions becomes a major bottleneck, often requiring extensive manual effort and reliance on institutional knowledge.

To address these limitations, there is a growing need for **intent-aware software engineering systems** that can bridge the gap between natural language requirements and code-level implementations. Such systems should be capable of semantically integrating heterogeneous development artifacts, automatically reconstructing requirement-to-code traceability, and providing contextual explanations of implementation decisions. Recent advances in **Agentic AI and large language models** offer a promising direction toward achieving this vision, enabling systems that can reason over software artifacts, infer latent relationships, and generate human-understandable insights.

This dissertation aims to address this gap by designing a **cloud-hosted agentic AI system** that integrates with enterprise tools and provides real-time, context-aware explanations of software intent. By embedding such intelligence directly within developer workflows, the proposed solution seeks to transform software understanding from a manual, fragmented process into an automated, semantically enriched experience.

Problem Statement

Modern software engineering environments are characterized by the extensive use of distributed development tools and collaborative workflows, resulting in the generation of large volumes of heterogeneous artifacts such as requirement specifications, issue tracking records, commit histories, pull request discussions, and documentation. While these artifacts collectively capture the evolution of a software system, they remain inherently **fragmented across multiple platforms**, lacking a unified semantic representation. This fragmentation of software knowledge creates significant barriers to understanding the rationale behind implementation decisions, especially in large-scale and long-lived codebases.

A critical consequence of this fragmentation is the **absence of effective requirement-to-code traceability**. Although development artifacts such as JIRA tickets and version control commits are implicitly related, explicit and reliable mappings between high-level requirements and corresponding code changes are rarely maintained. Existing approaches either rely on manual documentation, which is often incomplete or outdated, or employ static analysis techniques that fail to capture the underlying semantic intent. As a result, developers struggle to determine how specific requirements, bug reports, or feature requests are realized within the codebase.

This lack of traceability further exacerbates challenges in **developer onboarding, debugging, and system maintenance**. New contributors and engineers are required to invest substantial effort in navigating multiple sources of information to reconstruct context, leading to increased cognitive load and reduced productivity. Similarly, during debugging or feature enhancement, the inability to quickly understand the intent behind prior implementations results in inefficiencies and a higher risk of introducing regressions.

The problem is even more pronounced in **open-source software ecosystems**, where contributors often lack access to institutional knowledge and rely solely on publicly available artifacts. In such scenarios, incomplete documentation and disconnected development histories make it difficult to comprehend the purpose and evolution of code, thereby limiting effective contribution and long-term sustainability.

Despite advancements in artificial intelligence and software analytics, there remains a significant gap in systems that can **semantically integrate diverse development artifacts and reconstruct the underlying intent of software implementations**. Therefore, there is a need for an intelligent, scalable, and automated solution capable of bridging the gap between requirements and code, reducing knowledge fragmentation, and enabling context-aware understanding of software systems.

Dissertation Objectives

The primary objective of this dissertation is to design and develop an intelligent, cloud-hosted system capable of reconstructing the semantic intent behind software implementations in production-grade codebases. The specific objectives of the work are as follows:

- **To investigate and address the problem of semantic intent loss in software systems** by analysing how implementation rationale is fragmented across development artifacts such as requirement tickets, commit histories, and pull request discussions.
- **To develop a semantic reconstruction framework** that establishes meaningful relationships between natural language requirements and corresponding code changes, thereby enabling accurate requirement-to-code traceability.
- **To design and implement a cloud-hosted agentic AI system** that leverages multi-agent reasoning and large language models to infer, synthesize, and explain the underlying intent of software implementations.
- **To integrate the proposed system within modern Integrated Development Environments (IDEs)**, providing developers with real-time, context-aware assistance for understanding code, navigating traceability links, and interpreting implementation decisions.
- **To enable automated traceability workflows**, including mapping requirements to commits, identifying relevant code segments, and generating contextual explanations of how and why specific changes address requirements or defects.
- **To support issue reproduction and validation mechanisms**, allowing developers to understand the conditions under which a problem occurs and verify whether a given implementation successfully resolves the intended requirement or defect.
- **To evaluate the effectiveness of the proposed system** in reducing developer cognitive load, improving onboarding efficiency, and enhancing debugging and maintenance workflows in both enterprise and open-source environments.

System Architecture

The proposed system architecture is designed as a **cloud-hosted, agentic AI framework** for reconstructing software intent and establishing requirement traceability in production-grade codebases. The architecture follows a layered and modular design, enabling the system to ingest heterogeneous software engineering artifacts, extract semantically meaningful knowledge, reason over requirement-to-code relationships, and deliver contextual assistance directly within the developer's coding environment.

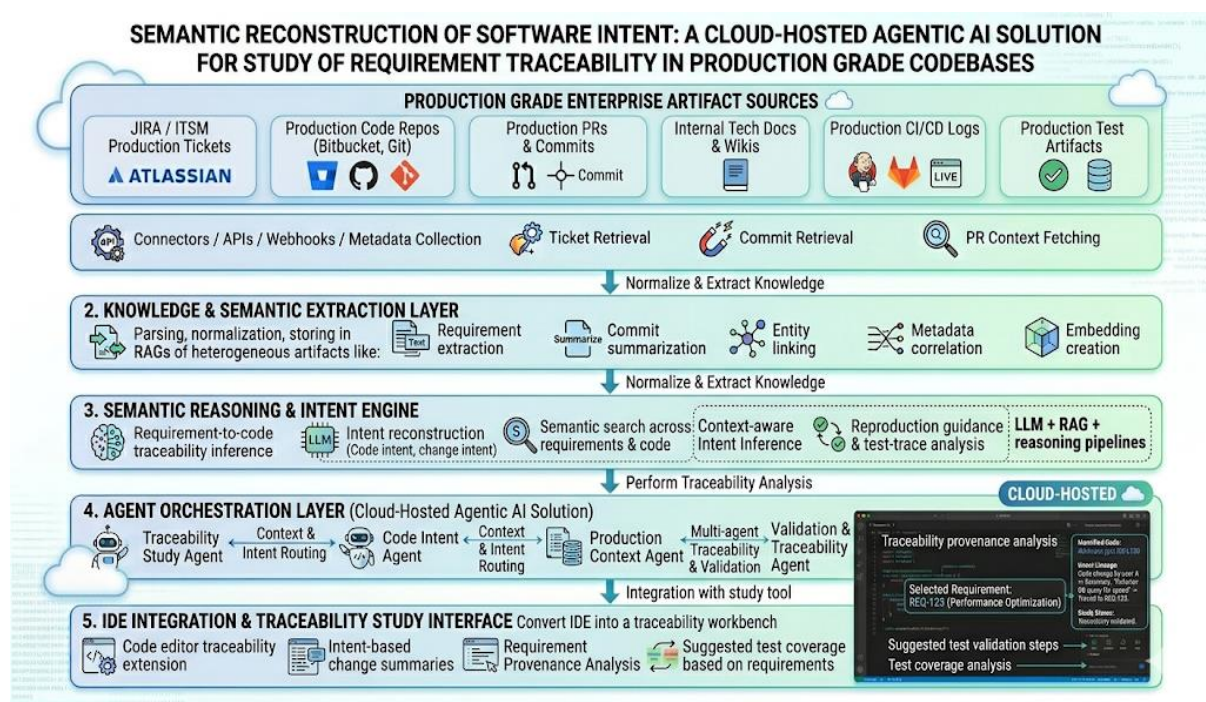
At a high level, the system acts as an intelligent bridge between enterprise development ecosystems and software developers. It collects contextual information from platforms such as **JIRA, Github, and related DevOps artifacts**, processes and semantically enriches this information through AI-driven reasoning, and surfaces intent-aware explanations inside the IDE. The architecture is cloud-hosted to ensure scalability, centralized intelligence, and seamless integration with distributed software teams.

➔ High-Level Architecture

The proposed architecture consists of five major subsystems:

1. Artifact Ingestion Layer
2. Knowledge Extraction Layer
3. Semantic Reasoning Engine
4. Agent Orchestration Layer
5. IDE Integration Layer

➔ Architecture Diagram



➔ Artifact Ingestion Layer

The **Artifact Ingestion Layer** is responsible for collecting software lifecycle artifacts from enterprise development tools. This includes data acquisition from **JIRA or ITSM systems for requirements, incidents, and defect tickets**, as well as **Bitbucket repositories for commits, branches, code history, and pull requests**. Additional contextual sources such as documentation repositories may also be incorporated.

This layer establishes secure and structured access to external systems through APIs, connectors, or event-driven hooks. Its primary role is to ensure that heterogeneous inputs are consistently captured and made available to downstream intelligence modules. Since software intent is distributed across multiple tools, this layer forms the foundational entry point for semantic reconstruction.

➔ Knowledge Extraction and Ingestion Layer

The **Knowledge Extraction Layer** transforms raw software engineering artifacts into machine-processable semantic representations. At this stage, development artifacts are parsed, normalized, and enriched with contextual metadata. For example, issue descriptions may be analyzed to identify requirement entities, commit messages may be summarized to capture implementation actions, and pull request discussions may be processed to extract rationale and review context.

This layer orchestrates the creation of semantic embeddings for all development artifacts, transforms them into dense vector representations, and stores them in a vector database leveraged by the RAG pipeline, enabling efficient ANN indexing, semantic similarity matching, and downstream LLM-based contextual reasoning.

The purpose of this layer is to convert fragmented textual and structural data into a unified knowledge substrate.

➔ Semantic Reasoning Engine

The **Semantic Reasoning Engine** is the core intelligence layer of the architecture. It is responsible for reconstructing the latent intent behind software changes by performing **cross-artifact correlation**, combining traceability inference, contextual semantic search, and AI-driven explanation generation. This engine analyses the relationships between requirements and source code changes to infer why a modification was introduced, what defect or requirement it addresses, and how the implementation aligns with the original need.

The reasoning engine leverages **large language models, retrieval pipelines, and semantic similarity mechanisms** to interpret both natural language requirements and code-level evolution. It is also expected to support advanced capabilities such as:

- requirement-to-code mapping,
- implementation rationale generation,
- reproduction guidance for defects,
- and validation-oriented reasoning for testing whether the issue has been adequately resolved.

This layer is the principal mechanism through which raw engineering artifacts are elevated into actionable developer knowledge.

➔ Agent Orchestration Layer

The **Agent Orchestration Layer** enables the system to operate as an **agentic AI framework** rather than a single monolithic reasoning engine. In this design, specialized agents collaborate to solve different sub-problems of software understanding.

For example:

- a **Traceability Agent** identifies requirement-to-code links,
- a **Code Intent Agent** explains why a change exists,
- a **Reproduction Agent** derives steps to recreate a reported issue,
- and a **Validation/Test Agent** suggests how the fix can be verified.

A coordinating agent manages context routing, decomposition of user queries, inter-agent communication, and synthesis of final responses. This modular design improves extensibility, explainability, and task specialization, making the system suitable for complex production engineering environments.

➔ IDE Integration Layer

The final layer is the **IDE Integration Layer**, through which the reconstructed software intent is delivered directly to developers inside their coding workflow. This may be implemented as a plugin or extension for a modern code editor. The goal of this layer is to present contextual insights without forcing developers to manually search across multiple external platforms.

Inside the IDE, the system can provide:

- explanations of why a method or module was introduced,
- links to the originating requirement or ticket,
- summaries of associated commits and pull requests,
- issue reproduction guidance,
- and testing or validation recommendations.

This transforms the IDE into an **intent-aware development environment**, where software understanding is augmented by semantically reconstructed historical and operational context.

➔ Architectural Significance

The proposed architecture is significant because it shifts software engineering support from a **document-centric retrieval model** to an **intent-centric reasoning model**. Instead of requiring developers to manually inspect JIRA issues, commit logs, and repository history, the system synthesizes these artifacts into a coherent explanation of software purpose and evolution.

From an enterprise standpoint, this architecture directly addresses major operational concerns such as:

- developer onboarding delays,
- knowledge transfer inefficiencies,
- debugging complexity,
- and poor traceability across fast-evolving codebases.

From a research standpoint, it provides a practical realization of **semantic software intent reconstruction using cloud-hosted agentic AI**, thereby forming the central technical contribution of this dissertation.

➔ Midterm Status of Implemented Architecture

At the midterm stage, the high-level architecture has been defined and the core system decomposition has been established in terms of ingestion, extraction, reasoning, orchestration, and IDE delivery. Initial work has been initiated on the artifact ingestion and knowledge extraction components, while the semantic reasoning and multi-agent orchestration modules are being progressively developed. This architectural foundation provides the basis for the remaining implementation and evaluation phases of the dissertation.

Progress of Prototype Implementation

The implementation of the proposed system has progressed in alignment with the planned phases outlined in the dissertation schedule, particularly covering the **artifact integration layer** and initial stages of **semantic processing**. The work completed so far focuses on establishing the foundational pipeline required for semantic reconstruction of software intent, including data acquisition, preprocessing, embedding generation.

➔ Completed Work

Significant progress has been made in developing the core infrastructure required for the system:

1) Integration with Development Artifacts

- Initial integration mechanisms have been established to interact with **JIRA (ITSM systems)** and **GitHub repositories**.
- APIs and connectors have been utilized to retrieve:
 - issue/ticket data,
 - commit histories,
 - and associated metadata.
- Basic linkage between tickets and commits has been successfully established using identifiers and metadata correlations.

2) Data Extraction and Processing Pipelines

- A structured pipeline has been implemented for:
 - ingesting development artifacts,
 - parsing textual and code-related information,
 - and normalizing heterogeneous data formats.
- Commit messages, ticket descriptions, and pull request context are transformed into unified representations for downstream processing.
- Additionally, the system incorporates **semantic embedding generation**, where:
 - all artifacts are converted into vector representations,
 - and stored in a **vector database**.
- This enables:
 - semantic similarity search,
 - intelligent retrieval,
 - and contextual augmentation using a **Retrieval-Augmented Generation (RAG)** approach.

➔ Work in Progress

The following components have been partially implemented and are currently under refinement:

1) Semantic Linking of Requirements to Code

- Initial methods for linking requirements (JIRA tickets) to code changes have been explored using:
 - metadata matching (e.g., ticket IDs in commits),
 - and embedding-based semantic similarity.
- While early results are promising, further improvements are required to enhance:
 - accuracy,
 - robustness,
 - and handling of complex, indirect relationships.

➔ Pending Work

The following major components are yet to be implemented and will form the focus of the next development phase:

1) Semantic Reasoning Pipeline

- Reasoning workflows to:
 - interpret requirement descriptions,
 - and associate them with relevant commits and code segments.
- The system will be capable of generating **basic “why” explanations** for all the cases.
- Perform deeper intent inference.
- Handle complex multi-artifact dependencies.
- Generate high-quality, explainable outputs.

2) Full Agentic AI Orchestration (Advanced)

- Development of a complete **multi-agent architecture**, including:
 - Traceability Agent,
 - Code Intent Agent,
 - Reasoning Agent
- Implementation of coordination logic for:
 - Task Decomposition,
 - Response Synthesis

3) IDE Integration (Advanced)

- Development of a fully functional **IDE plugin/extension** to:
 - provide real-time intent-aware assistance,
 - display requirement-to-code mappings,
 - and deliver contextual explanations within the developer workflow.

➔ Alignment with Plan of Work

The current implementation progress aligns closely with the defined project timeline, particularly the completion of:

- **Requirement Analysis and System Architecture Design phase**
- **Prototype Development Phase I (Artifact Integration Layer)**

The upcoming work will focus on:

- **Prototype Development Phase II (Agentic AI reasoning module)**

Plan of Work

Phases	Start Date- End Date	Work to be done	Status
Dissertation Outline & Proposal Finalization	20 Jan 2026 – 07 Feb 2026	Literature survey on requirement traceability, semantic code understanding, and agentic AI. Finalization of dissertation outline and problem formulation. Submission of outline.	COMPLETED
Requirement Analysis & System Architecture Design	08 Feb 2026 – 28 Feb 2026	Design of the cloud-hosted agentic framework. Identification of integration requirements with ITSM, Code Repositories, and DevOps artifacts. Definition of semantic reconstruction pipeline.	COMPLETED
Prototype Development Phase I (Artifact Integration Layer)	01 Mar 2026 – 20 Mar 2026	Implementation of tools for issue tracking and version control systems. Extraction of commits, tickets, pull request context, and traceability of metadata.	COMPLETED
Midterm Report Preparation & Submission	21 Mar 2026 – 28 Mar 2026	Documentation of progress, architecture, preliminary results, and prototype status. Submission of midterm dissertation report.	COMPLETED
Prototype Development Phase II (Agentic AI Reasoning Module)	29 Mar 2026 – 20 Apr 2026	Development of multi-agent semantic reasoning for intent reconstruction. Implementation of conversational explanations and requirement-to-code mapping generation.	PENDING
Testing, Validation & User Evaluation	21 Apr 2026 – 05 May 2026	Functional testing of the system on production-grade repositories. Evaluation of onboarding acceleration, cognitive load reduction, and traceability accuracy.	PENDING
Final Dissertation Writing & Review	06 May 2026 – 11 May 2026	Consolidation of findings, final documentation, proofreading, and incorporation of supervisor feedback. Completion of final dissertation manuscript.	PENDING
Final Submission	12 May 2026	Final review and official submission of the dissertation report.	PENDING

Observations (Preliminary - Midterm)

At the current stage of implementation, the system has produced early insights primarily from the **artifact ingestion and knowledge extraction layers**. While full semantic reasoning and intent reconstruction are not yet implemented, the initial results demonstrate promising capabilities in handling and organizing development artifacts.

➔ Improved Accessibility of Development Artifacts:

The system successfully consolidates data from JIRA and GitHub into a unified pipeline, reducing the need to manually navigate across multiple tools.

➔ Reduction in Manual Lookup Effort:

Using embedding-based retrieval, relevant commits and related artifacts can be identified more efficiently compared to traditional keyword-based search.

➔ Effectiveness of Semantic Embeddings:

The use of vector embeddings enables context-aware retrieval of related artifacts, even in cases where direct references (such as ticket IDs) are absent.

➔ Structured Representation of Knowledge:

The transformation of raw artifacts into normalized and embedded formats provides a structured foundation for further semantic analysis and reasoning.

➔ Dependency on Data Quality:

The effectiveness of retrieval and linkage is influenced by:

- clarity of ticket descriptions,
- quality of commit messages,
- and consistency in metadata usage.

These preliminary observations indicate that the implemented layers are capable of **efficient data ingestion, semantic representation, and retrieval**, forming a strong foundation for the subsequent development of **reasoning, traceability mapping, and intent reconstruction capabilities**.

Challenges

During the implementation of the proposed system, several challenges were encountered, primarily due to the inherent complexity of software development artifacts and the semantic gap between natural language requirements and code-level implementations. These challenges highlight the non-trivial nature of reconstructing software intent in real-world environments.

➔ Linking Natural Language Requirements to Code

One of the primary challenges lies in establishing meaningful relationships between **natural language requirements (e.g., JIRA tickets)** and **source code changes**. Requirements are often written in high-level, informal language, whereas code represents low-level, structured logic. Bridging this gap requires robust semantic understanding, which is difficult to achieve using simple keyword matching or shallow similarity techniques.

Even with embedding-based approaches, accurately mapping requirements to relevant code segments remains challenging, particularly in cases involving:

- abstract or ambiguous requirement descriptions
- indirect implementation spread across multiple commits
- and lack of explicit references between artifacts.

➔ Noisy and Inconsistent Development Artifacts

The quality of development artifacts significantly affects system performance. In practice, artifacts such as:

- commit messages,
- ITSM/JIRA ticket descriptions,
- and pull request discussions

are often **incomplete, inconsistent, or poorly structured**.

Examples include:

- vague commit messages (e.g., “minor fix”, “update code”),
- missing or unclear requirement descriptions,
- and lack of standardized documentation practices.

This noise introduces ambiguity and reduces the effectiveness of both metadata-based linking and semantic retrieval techniques.

➔ Incomplete or Missing Traceability

A major challenge is the **absence of explicit traceability links** between requirements and code. In many cases:

- commits do not reference ticket IDs,
- multiple commits contribute to a single requirement,
- or a single commit addresses multiple issues.

This leads to fragmented and incomplete traceability, making it difficult to establish reliable mappings. As a result, the system must rely on indirect signals such as semantic similarity, which may not always be accurate.

➔ Heterogeneity of Data Sources

The system integrates data from multiple platforms (e.g., JIRA, GitHub), each with different structures, formats, and conventions. Handling this heterogeneity requires:

- normalization of data formats,
- consistent parsing strategies,
- and alignment of metadata across sources.

Ensuring consistency while preserving contextual meaning across these diverse artifacts is a non-trivial task.

The challenges encountered during implementation highlight the complexity of building an intent-aware software engineering system. Issues such as **semantic mismatch, noisy data, incomplete traceability** emphasize the need for advanced reasoning mechanisms and structured approaches. Addressing these challenges forms the basis for the next phase of development, particularly in enhancing semantic reasoning, agentic AI orchestration, and traceability accuracy.

Conclusion (Midterm)

At the midterm stage, the dissertation has successfully established the foundational components required for semantic reconstruction of software intent. The work completed so far includes the design of the overall system architecture and the implementation of the **artifact ingestion and knowledge extraction layers**, enabling the collection, normalization, and semantic representation of development artifacts from platforms such as JIRA and GitHub. Additionally, the integration of embedding-based retrieval using vector databases demonstrates the system's capability to organize and retrieve contextually relevant information from heterogeneous sources.

These developments validate the **technical feasibility** of the proposed approach. The ability to unify distributed software artifacts and perform semantic retrieval confirms that the system can serve as a foundation for higher-level functionalities such as requirement-to-code traceability and intent inference. Although advanced reasoning and agentic orchestration are not yet implemented, the current progress establishes a robust base for these capabilities in subsequent phases.

The expected impact of this work is significant in both enterprise and open-source software engineering contexts. By enabling intent-aware understanding of codebases, the system has the potential to:

- reduce developer cognitive load,
- accelerate onboarding and knowledge transfer,
- improve debugging and maintenance efficiency,
- and enhance requirement traceability across evolving systems.

Overall, the midterm progress demonstrates that the proposed solution is both **practically viable and academically relevant**, providing a strong foundation for further development toward a complete agentic AI system for semantic software understanding.